

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ**

ПЕНЗЕНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ

Кафедра «Математическое обеспечение и применение ЭВМ»

Курсовая работа

по дисциплине «Структуры и алгоритмы обработки данных»

на тему «Разработка библиотеки функций для поддержки работы с бинарными деревьями
поиска»

ПГУ 09.03.04 - 03КР232.22 ПЗ

Направление подготовки – 09.03.04 Программная инженерия

Профиль подготовки – Программная инженерия

Выполнил студент

Группа

Осин С. М.

23ВП2

Руководитель

к.т.н., доцент

Самуйлов С.В.

Работа защищена с оценкой

Преподаватели

Дата защиты

2024

ПЕНЗЕНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ

Кафедра «Математическое обеспечение и применение ЭВМ»

«УТВЕРЖДАЮ»

Зав. кафедрой  А.Ю. Козлов
« ____ » _____ 2024 г.

ЗАДАНИЕ

на курсовое проектирование по курсу
«Структуры и алгоритмы обработки данных»

Студенту Осину С.М. Группа 23ВП2
Тема проекта Разработка библиотеки функций для поддержки работы с бинарными деревьями поиска

Исходные данные (технические требования) на проектирование

Должны быть разработаны следующие функции: создать пустое бинарное дерево поиска, добавить новое значение, удалить значение, модифицировать значение, поиск значение, обход слева направо, обход сверху вниз, обход снизу вверх.

Выполнить оценку временных характеристик работы алгоритма сортировки прямым обменом и сортировки с помощью бинарного дерева поиска.

Данные отобразить в табличной и графической формах.

Для разработки приложений использовать по выбору одну из сред визуального программирования, Visual Studio, Borland Builder C++, Lazarus, MS Visual C++, Borland Delphi и т.п.

Пользовательский интерфейс должен обеспечивать привычное и удобное представление информации, простое и эффективное выполнение основных функций приложения.

Программное обеспечение должно быть полностью отлажено и протестировано, функционировать под управлением ОС Windows.

Объем работы по курсу

1. Расчетная часть

1. Анализ требований к разработке программного обеспечения
2. Проектирование программы
3. Разработка программного обеспечения
4. Тестирование программного обеспечения

2. Графическая часть

1. Диаграмма вариантов использования
2. Диаграмма классов
3. Диаграмма деятельности
4. Диаграмма компонентов

3. Экспериментальная часть

1. Подготовка набора тестовых данных
2. Отладка и тестирование программного обеспечения

Срок выполнения проекта по разделам

1. Анализ требований к разработке ПО	к	15.10.24
2. Проектирование программы	к	01.11.24
3. Разработка программного обеспечения	к	01.12.24
4. Тестирование программного обеспечения	к	15.12.24
5. Оформление пояснительной записки	к	28.12.24
6. Защита курсовой работы	к	29.12.24
	к	

Дата выдачи задания « 10 » сентября 2024

Дата защиты проекта « » _____

Руководитель Сам /Самуйлов С.В./

Задание получил «10» сентября 2024 г.

Студент Осин Осин С.М.

Реферат

Пояснительная записка содержит 45 листов, 19 рисунков, 4 таблицы, 6 использованных источников, 1 приложение.

СОРТИРОВКА, СОРТИРОВКА ПРЯМЫМ ОБМЕНОМ, СОРТИРОВКА С ПОМОЩЬЮ БИНАРНОГО ДЕРЕВА ПОИСКА.

Объектом исследования являются наиболее известные алгоритмы сортировки прямым обменом и с помощью бинарного дерева поиска.

Целью курсовой работы является разработка приложения для оценки временных характеристик работы алгоритма сортировки прямым обменом и сортировки с помощью бинарного дерева поиска.

Разработка проводилась на языке программирования C# в среде программирования Visual Studio.

Осуществлено функциональное тестирование разработанного программного обеспечения, которое показало корректность его работы.

					ПГУ 09.03.04 - 03КР232.22 ПЗ			
Изм.	Лист	№ докум.	Подп.	Дата				
Разраб.		Осин С. М.			«Разработка библиотеки функций для поддержки работы с бинарными деревьями поиска».	Лит.	Лист	Листов
Пров.		Самуйлов С.В.					4	45
						Группа 23ВП2		

Содержание

Введение	6
1 Постановка задачи и анализ предметной области	8
1.1 Основные понятия и определения	8
1.2 Постановка задачи на разработку программы.....	9
1.3 Анализ требований.....	10
1.3.1 Требования к интерфейсу пользователя	10
1.3.2 Требования к структуре данных	11
1.3.3 Требования к программным средствам.....	13
1.4 Технологии разработки программного обеспечения	16
2 Проектирование программы.....	19
2.1 Модель интерфейса	19
2.2 Проектирование структур данных	20
2.3 Структура программного обеспечения	20
3 Реализация программы	24
3.1 Кодирование	24
3.2 Диаграмма компонентов.....	25
4 Тестирование.....	26
4.1 Виды тестирования программных средств	26
4.2 Функциональное тестирование программы	28
5 Анализ результатов.....	34
Заключение	36
Список использованных источников	37
Приложение А	38

Введение

Актуальность сравнения сортировок бинарным деревом поиска (BST) и прямым обменом (методом пузырька) обусловлена фундаментальным значением этих алгоритмов в изучении и понимании принципов сортировки данных. Несмотря на то, что метод пузырька считается наивным и малоэффективным для больших массивов данных, его простота делает его ценным инструментом для обучения основам алгоритмов сортировки. Сравнение с бинарным деревом поиска, более сложным, но потенциально эффективным алгоритмом, позволяет продемонстрировать преимущества использования более совершенных структур данных и алгоритмических подходов для решения задачи сортировки. Это сравнение наглядно иллюстрирует компромисс между простотой реализации и эффективностью работы алгоритма.

Сравнение этих двух алгоритмов актуально для иллюстрации принципов анализа сложности алгоритмов. Метод пузырька имеет квадратичную временную сложность $O(n^2)$ в худшем и среднем случаях, что делает его непригодным для больших объемов данных. В то же время бинарное дерево поиска в лучшем случае обеспечивает логарифмическую сложность $O(n \log n)$, а в худшем — линейную сложность $O(n)$, что зависит от сбалансированности дерева.

Таким образом, сравнение сортировки бинарным деревом поиска и прямым обменом является ценным учебным инструментом для понимания принципов сортировки, анализа сложности алгоритмов и выбора подходящего алгоритма в зависимости от размера данных и требований к производительности. Простота метода пузырька позволяет легко понять основы сортировки, а сравнение с БДП демонстрирует преимущества использования более сложных, но эффективных алгоритмов и структур данных.

Целью разрабатываемого приложения является оценка временных характеристик работы двух алгоритмов: прямой обмен (пузырьковый метод) и с помощью бинарного дерева поиска. С помощью табличных и графических формам проведём сравнение эффективности каждого из алгоритмов.

Для достижения поставленной цели разобьём разработку программного обеспечения на несколько частей для снижения рисков, упрощения процесса достижения цели и повышения эффективности.

Сначала проведём анализ предметной области, а именно проанализируем структуры данных, описанные в техническом задании: массив и бинарное дерево поиска. Затем проведём анализ требований для интерфейса для удобства использования его пользователем и для структур данных, чтобы в процессе сортировок не возникало ошибок. Спроектируем диаграмму вариантов использования для описания функционала разработанного программного обеспечения. Далее выберем технологию разработки программного обеспечения для эффективного планирования проектирования и создания программного продукта.

После того, как проанализируем предметную область и требования, разработаем форму, обеспечивающую функционал, описанный в техническом задании. Далее запрограммируем структуры данных (бинарное дерево поиска и массив) для дальнейшей работы с ними. Затем спроектируем диаграмму классов для отображения ключевых аспектов системы (классов) и их взаимодействия.

Для анализа одного из видов сортировки спроектируем диаграмму деятельности (для сортировки прямым обменом), описывающая порядок выполнения алгоритма и диаграмма компонентов, в которой описаны файлы, используемые в проекте, и их взаимодействие.

После окончания разработки программного обеспечения для сравнения алгоритмов сортировки выберем один из видов тестирования и проведём его с разработанным программным обеспечением.

По окончании тестирования, подведём итоги тестирования и разработанного программного обеспечения: проанализируем какой алгоритм сортировки лучше использовать и в каких типах упорядоченности массива.

Затем сформулируем общее заключение, исходя из работы программного обеспечения и сравнения алгоритмов сортировки.

Для разработки программного обеспечения будет использована среда визуального программирования – Visual Studio 2022. Язык программирования – C#. Microsoft .NET Framework – базовая программная платформа, необходимая для разработки приложений на C#. Программное обеспечение будет полностью отлажено и протестировано, функционировать под управлением Операционной Системы Windows.

1 Постановка задачи и анализ предметной области

1.1 Основные понятия и определения

Алгоритмы — это набор чётко определённых инструкций или шагов, которые выполняются в определённом порядке для решения конкретной задачи или достижения определённой цели. [1]

Примеры алгоритмов в программировании включают в себя сортировку массивов, поиск элементов в массиве, вычисление факториала числа, решение систем линейных уравнений и многие другие задачи.

Алгоритмы используются в качестве основы для создания программного обеспечения, так как они позволяют программистам структурировать и организовать код таким образом, чтобы он был понятным, эффективным и надёжным.

Структуры данных — это способы организации, хранения и управления данными в компьютере. В общем случае структура данных определяет, как данные организованы в памяти компьютера и как к ним можно получить доступ. Структуры данных используются для хранения и организации данных таким образом, чтобы их можно было эффективно использовать в различных операциях, таких как поиск, вставка, удаление и сортировка.

Сортировка — это процесс организации элементов в коллекции (например, массиве или списке) в определённом порядке, таком как по возрастанию или убыванию. В результате сортировки элементы располагаются в нужном порядке, что упрощает их обработку и поиск.

Существует множество алгоритмов сортировки, каждый из которых имеет свои преимущества и недостатки в зависимости от размера и характера данных, с которыми работает программа.

Сортировка методом прямого обмена (Bubble Sort) — это простой алгоритм сортировки, который работает путем многократного прохода через список, сравнивая каждый элемент со своим соседом и меняя их местами, если они находятся в неправильном порядке. Алгоритм называется "пузырьковой" сортировкой, потому что более крупные элементы "всплывают" к вершине списка, подобно тому, как пузырьки всплывают в воде. [1]

При первом проходе алгоритм просматривает всю последовательность размера n и помещает наибольший элемент в конец. При втором проходе алгоритм просматривает уже $(n-1)$ элемент и помещает следующий по величине элемент в предпоследнюю позицию и т. д.

Алгоритм выполняется до тех пор, пока за один проход не будет совершено ни одной перестановки. В таком случае считается, что множество отсортировано, и процесс успешно завершён. [1]

Бинарное дерево — это структура данных, в которой каждый узел имеет не более двух детей, называемых левым и правым. В каждом узле хранятся данные, а также ссылки на левого и правого ребенка. Бинарные деревья используются для быстрого поиска, добавления и удаления элементов, а также для организации данных в иерархическом порядке. Бинарные деревья имеют

множество применений, таких как хранение и организация данных в базах данных, файлах и кэшах, а также в алгоритмах поиска, сортировки и графов. [2]

Бинарное дерево поиска (BST) — это тип бинарного дерева, в котором каждый узел содержит ключ, и все ключи в левом поддереве меньше, чем ключ в текущем узле, а все ключи в правом поддереве больше или равны ключу в текущем узле. Это свойство позволяет проводить быстрый поиск, добавление и удаление элементов в дереве. [2]

Основные операции, которые можно проводить на бинарном дереве поиска:

- Поиск (Search): для поиска элемента в BST, начиная с корня, сравниваем значение искомого элемента с ключом текущего узла. Если они равны, то мы нашли искомый элемент. Если искомый элемент меньше ключа текущего узла, переходим к левому поддереву, иначе переходим к правому поддереву. Повторяем процесс, пока не найдем искомый элемент или не дойдем до листа;
- Добавление (Insert): для добавления элемента в BST, сначала проводим поиск места, где должен находиться новый элемент. Если элемент уже существует в дереве, то ничего не делаем. Если элемент не существует, создаем новый узел с данным значением и вставляем его в дерево. Если новый узел меньше текущего узла, он становится левым ребенком текущего узла, иначе он становится правым ребенком текущего узла;
- Удаление (Delete): для удаления элемента из BST, сначала проводим поиск элемента в дереве. Если элемент не найден, то ничего не делаем. Если элемент найден, то удаляем его из дерева.

Существует несколько способов удаления узла из бинарного дерева поиска:

- Если узел не имеет детей, просто удаляем его из дерева;
- Если узел имеет только одного ребенка, заменяет узел на его ребенка;
- Если узел имеет двух детей, находим узел-предшественник (наиболее левый узел в правом поддереве) или узел-потомка (наиболее правый узел в левом поддереве), заменяем значение удаляемого узла на значение узла-предшественника или узла-потомка, а затем удаляем узел-предшественник или узел-потомок из дерева.

В среднем, операции поиска, добавления и удаления в бинарном дереве поиска занимают $O(\log n)$ времени. Однако в худшем случае, если дерево не сбалансировано, операции могут занимать $O(n)$ времени. [2]

1.2 Постановка задачи на разработку программы

Задача курсовой работы - выполнить оценку характеристик работы алгоритма сортировки прямого обмена и с помощью бинарного дерева поиска:

- Задать размер массива;
- Создать пустое бинарное дерево поиска;
- Добавить новое значение в дерево;
- Удалить значение из дерева;
- Модифицировать значение в дереве;
- Найти значение в дереве;
- Выполнить сортировку указанного пользователем размера массива.

Результаты временных характеристик алгоритмов сортировок будут представлены в графической и табличной формах.

Для разработки приложения будет использовано среду визуального программирования Visual Studio 2022.

Пользовательский интерфейс будет обеспечивать привычное и удобное представление информации, простое и эффективное выполнение основных функций приложения.

Программное обеспечение будет полностью отлажено и протестировано, функционировать под управлением Операционной Системы Windows.

1.3 Анализ требований

1.3.1 Требования к интерфейсу пользователя

Приложение «Osin_23VP2» должно представлять пользователю удобный интерфейс, позволяющий:

- Создать пустое бинарное дерево поиска;
- Задать размер массива.
- Добавить новое значение в дерево;
- Удалить значение из дерева;
- Найти значение в дереве;
- Изменить значение в дереве;
- Сортировать массив 2 методами сортировки: с помощью бинарного дерева поиска и прямым обменом.

Требования в приложении:

- Если пользователь задал размер массива ноль или меньше, то приложение выдаст ошибку “размер дерева должен быть строго больше нуля”;
- Если введенного пользователем значения нет в дереве, то результат работы приложения будет “значение отсутствует в бинарном дереве поиска”;

- Если заданного пользователем значение, которое он желает удалить, нет в дереве, приложение выдаст ошибку “Данное значение отсутствует в бинарном дереве поиска”;

1.3.2 Требования к структуре данных

Заданные структуры данных – бинарное дерево поиска (BST) и массив.

Бинарное дерево поиска – структура данных, в которой каждый узел содержит один ключ и два поддерева:

- Левое поддерево: все ключи в левом поддереве меньше, чем ключ в текущем узле;
- Правое поддерево: все ключи в правом поддереве больше или равны ключу в текущем узле.

Требования к структуре данных бинарного дерева поиска:

- Каждый узел имеет один ключ и два поддерева (левое и правое);
- Все ключи в левом поддереве меньше, чем ключ в текущем узле;
- Все ключи в правом поддереве больше или равны ключу в текущем узле;
- Все поддеревья также должны быть бинарными деревьями поиска;
- Дерево не может содержать повторяющиеся ключи.

Логическое представление бинарного дерева поиска представлено на рисунке 1.

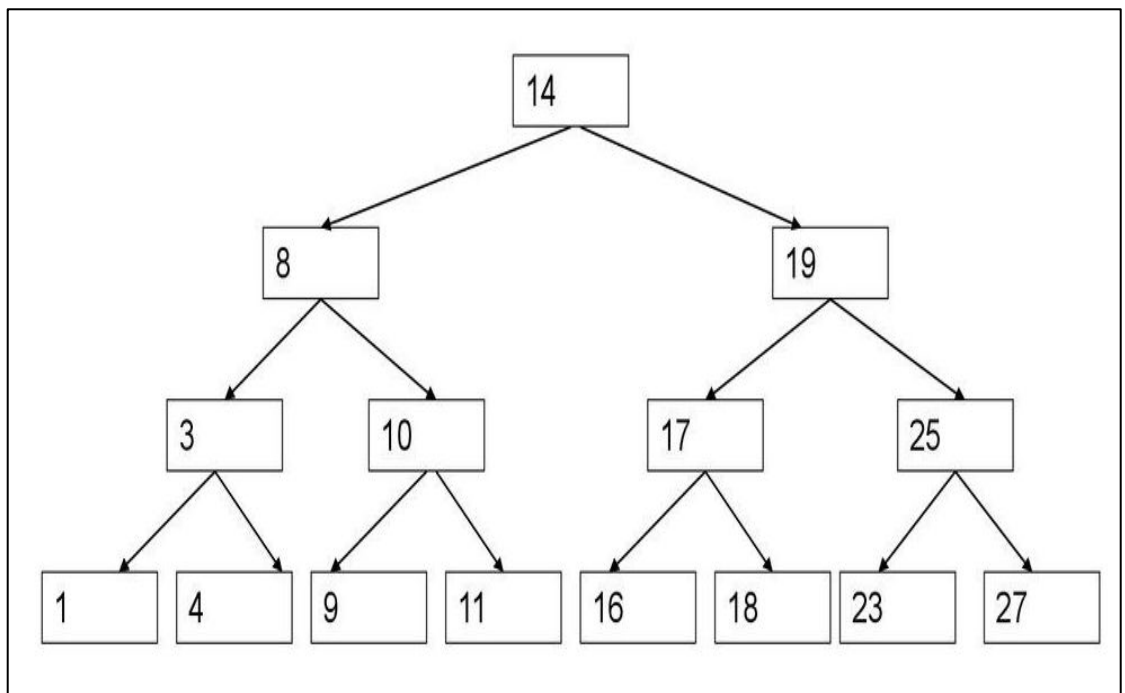


Рисунок 1 – Логическое представление бинарного дерева поиска

В данном бинарном дереве поиска у каждого его узла не более двух дочерних узлов, у каждого из которых тоже не более двух дочерних. Корнем дерева или же нулевой уровень - родитель дерева (в данном случае число 4).

А глубина дерева – максимальный уровень (в данном случае третий). Также глубину можно определить как длину самого длинного пути от корня до листа, если следовать от родительской вершины до потомка.

Основными операциями данного бинарного дерева поиска являются:

- Поиск значения;
- Вставка значения;
- Удаления значения;
- Модификация значения;
- Обход сверху вниз;
- Обход снизу вверх;

Массив (array) — это структуры данных, которые хранят набор элементов одного и того же типа в линейной последовательности. Каждый элемент в массиве имеет индекс, который уникально идентифицирует его положение в массиве. Индексы обычно начинаются с нуля, и первый элемент массива имеет индекс 0.

Особенностью массива как структуры данных является константная вычислительная сложность доступа к элементу массива по индексу. Массив относится к структурам данных с произвольным доступом.

Логическое представление массива представлено на рисунке 2.



Рисунок 2 – Логическое представление массива

Массивы могут быть использованы для хранения различных типов данных, таких как целые числа, вещественные числа, строки и даже другие массивы. Они полезны в ситуациях, когда требуется быстрый доступ к

элементам по их индексу или когда элементы нужно обрабатывать в определенном порядке.

1.3.3 Требования к программным средствам

Диаграмма вариантов использования (Use Case Diagram) — это визуальный инструмент, используемый в анализе и проектировании систем для описания функциональности системы с точки зрения пользователя. [3]

Диаграмма вариантов использования состоит из следующих элементов:

- Актёры: Сущности, которые взаимодействуют с системой. Они могут быть пользователями, другими системами или устройствами;
- Варианты использования: Функции или сценарии, которые система предоставляет акторам для достижения их целей;
- Связи: Стрелки, которые соединяют акторов с вариантами использования, показывая, какие акторы участвуют в вариантах использования;
- Система: Квадрат, который представляет саму систему, с которой взаимодействуют акторы.

Цели создания диаграммы вариантов использования:

- Определить общие границы и контекст моделируемой предметной области на начальных этапах проектирования системы;
- Сформулировать общие требования к функциональному поведению проектируемой системы;
- Разработать исходную концептуальную модель системы для её последующей детализации в форме логических и физических моделей;
- Подготовить исходную документацию для взаимодействия разработчиков системы с её заказчиками и пользователями.

В основном диаграммы вариантов использования используются в следующих случаях:

- Анализ требований: Диаграмма вариантов использования помогает понять и документировать требования пользователей к системе. Она показывает, какие функции должна выполнять система, чтобы соответствовать потребностям пользователей;
- Проектирование системы: Диаграмма вариантов использования используется для определения границ системы и для определения интерфейсов между системой и внешними акторами (например, пользователями или другими системами);
- Управление проектами: Диаграмма вариантов использования может использоваться для управления проектами, отслеживая прогресс реализации различных функций системы.

Диаграмма вариантов использования может использоваться в сочетании с другими видами диаграмм, такими как диаграммы классов, диаграммы последовательностей и диаграммы состояний, для полного описания системы. [3]

Функции приложения:

- Выполнить сортировку массива;
- Задать размер массива;
- Создать пустое дерево;
- Модифицировать значение в дереве;
- Найти значение в дереве;
- Добавить значение в дерево;
- Удалить значение из дерева;
- Отобразить результат.

Исходя из функций, была построена диаграмма вариантов использования, показанная на рисунке 2, которая раскрывает функционал приложения. [3]

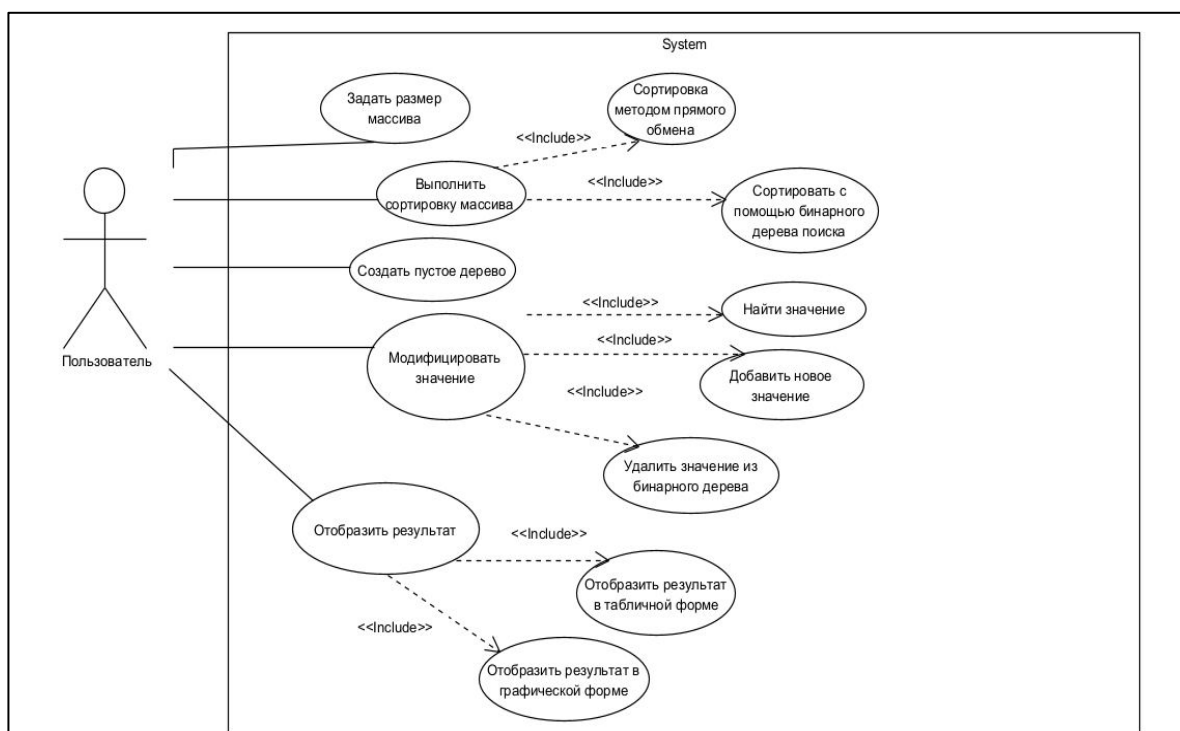


Рисунок 3 – Диаграмма вариантов использования

Исходя из вариантов использования, были построены таблицы (1–3) для описания спецификации прецедентов.

Таблица 1 – Сценарий варианта использования «Задать размер массива»

Наименование: задать размер массива
--

Продолжение таблицы 1

ID: 1
Краткое описание: пользователь задаёт значение размера массива
Действующие лица: пользователь
Предусловие: значение строго больше 0
Основной поток: 1. Пользователь задаёт значение размера массива. 1.1. Если значение меньше или равно нулю, то приложение выводит сообщение «Размер массива должен быть строго больше 0». 1.2. Иначе: 1.2.1. Берётся массив данным пользователем размером и сортируется.
Постусловие: массив создан

Таблица 2 – Сценарий варианта использования «Выполнить сортировку массива»

Наименование: выполнить сортировку массива
ID: 2
Краткое описание: пользователь нажимает на кнопку «Сортировать»
Действующие лица: пользователь
Предусловие: размер массива положительный
Основной поток: 1. Приложение сортирует массив двумя 2 способами: методом прямого обмена и с помощью бинарного дерева поиска. 2. Выводится время выполнения каждой из сортировки в табличной и графической форме.
Постусловие: массив отсортирован

Таблица 3 – Сценарий варианта использования «Модифицировать значение»

Наименование: модифицировать значение
ID: 4
Краткое описание: пользователь вводит значение, которое имеется в бинарном дереве и которое изменит значение.
Действующие лица: пользователь
Предусловие: значение имеется в дереве.

Основной поток:

1. Пользователь вводит значение.
 - 1.1. Если значения нет в дереве – приложение выводит сообщение «Данного значения нет в дереве»
 - 1.2. Иначе
 - 1.2.1. Пользователь вводит значение, которое будет вставлено в бинарное дерево поиска вместо старого.
 - 1.3. Приложение выводит сообщение «Значение изменено»

Постусловие: значение изменено

1.4 Технологии разработки программного обеспечения

Технология разработки программного обеспечения — это комплекс организационных мер, операций и приёмов, направленных на разработку программных продуктов высокого качества в рамках отведённого бюджета и в срок.

Одними из самых популярных технологий разработки ПО являются Agile, DevOps, Waterfall и Lean.

Agile - это один из самых популярных подходов к разработке программного обеспечения в современном мире. Он фокусируется на гибкости и адаптивности в процессе разработки, позволяя командам реагировать на меняющиеся требования и корректировать курс по мере необходимости. Agile включает в себя несколько методологий, таких как Scrum, Kanban, Extreme Programming (XP) и Crystal, каждая из которых имеет свой набор правил и практик. [4]

Scrum — это самая популярная методология Agile, которая включает в себя короткие итерации разработки, называемые спринтами, каждая из которых длится от одной до четырех недель. В каждой итерации команда разрабатывает набор функций или задач, определенный заказчиком, а затем тестирует и дорабатывает их перед переходом к следующей итерации. Scrum также включает в себя регулярные встречи, такие как ежедневные стэнд-апы, плановые собрания и ретроспективы, чтобы гарантировать, что команда работает эффективно и эффективно решает любые проблемы, которые могут возникнуть.

Agile также подчеркивает важность тесного сотрудничества между разработчиками, тестировщиками и заказчиками. Заказчик работает в тесном сотрудничестве с командой разработчиков, чтобы определить приоритеты и функции, которые должны быть разработаны в каждой итерации. Разработчики и тестировщики работают вместе, чтобы гарантировать, что программный продукт соответствует требованиям и работает правильно. Регулярный обмен информацией и обратная связь также являются неотъемлемой частью Agile, что

позволяет команде быстро реагировать на меняющиеся требования и корректировать курс по мере необходимости. [4]

DevOps - это еще один популярный подход к разработке программного обеспечения в современном мире. Он сочетает в себе практики разработки (Dev) и операций (Ops) для ускорения процесса разработки, развертывания и управления приложениями. DevOps фокусируется на автоматизации процессов, таких как сборка, тестирование и развертывание кода, чтобы ускорить и упростить процесс разработки. Он также включает в себя мониторинг и устранение неполадок в рабочей среде, чтобы гарантировать стабильность и надежность приложения. DevOps подчеркивает важность сотрудничества между командами разработчиков и операторов, а также обмена информацией и знаниями между ними. Одним из ключевых аспектов DevOps является использование инфраструктуры как кода (IaC), которая позволяет командам автоматизировать настройку и управление инфраструктурой приложения. IaC позволяет командам создавать и управлять инфраструктурой в коде, что гарантирует повторяемость и предсказуемость в процессе развертывания. DevOps также включает в себя использование инструментов непрерывной интеграции и непрерывной доставки (CI/CD), которые автоматизируют процесс сборки, тестирования и развертывания кода.

Waterfall — это традиционный подход к разработке программного обеспечения, который все еще используется в некоторых организациях. Он следует строгой последовательности этапов, таких как сбор требований, проектирование, реализация, тестирование и развертывание. Каждый этап должен быть завершен полностью, прежде чем переходить к следующему. Waterfall был разработан в 1970-х годах и был широко принят в течение нескольких десятилетий, прежде чем быть замененными более гибкими подходами, такими как Agile.

Waterfall подходит для проектов, имеющих четко определенные требования и жесткие сроки, но может быть менее гибким для проектов, требующих большей адаптивности или меняющихся требований. Он также может быть более медленным и менее эффективным, чем другие подходы, поскольку каждый этап должен быть завершен полностью, прежде чем переходить к следующему.

Lean — это подход к разработке программного обеспечения, основанный на принципах бережливого производства. Он был разработан в 1990-х годах на основе принципов Toyota Production System (TPS) и был применен к разработке программного обеспечения в начале 2000-х годов. Lean фокусируется на минимизации потерь и оптимизации процессов, чтобы обеспечить максимальную ценность для клиента.

Lean включает в себя несколько принципов, таких как удаление ненужных этапов и документов, сокращение времени цикла разработки, фокусирование на непрерывном обучении и совершенствовании, и сотрудничество и обмен информацией между всеми участниками процесса

разработки. Lean также подчеркивает важность понимания ценности для клиента и фокусирования на ней в процессе разработки.

Одним из ключевых аспектов является использование визуализации процессов, чтобы сделать их более понятными и управляемыми. Визуализация процессов позволяет командам увидеть, где находятся задержки и потери, и принять меры для их устранения. Lean также включает в себя использование инструментов управления очередью, таких как Kanban, чтобы гарантировать, что команда работает эффективно и не перегружена работой.

Исходя из вышеперечисленных технологий разработки программного обеспечения, в курсовой работе будет использована технология Agile, потому что данный подход направлен на повышение гибкости, коммуникации, производительности, качества и удовлетворенности заказчика. [4]

2 Проектирование программы

2.1 Модель интерфейса

В курсовой работе был разработан пользовательский интерфейс, включающий в себя одну форму, показанная на рисунке 4. [5]

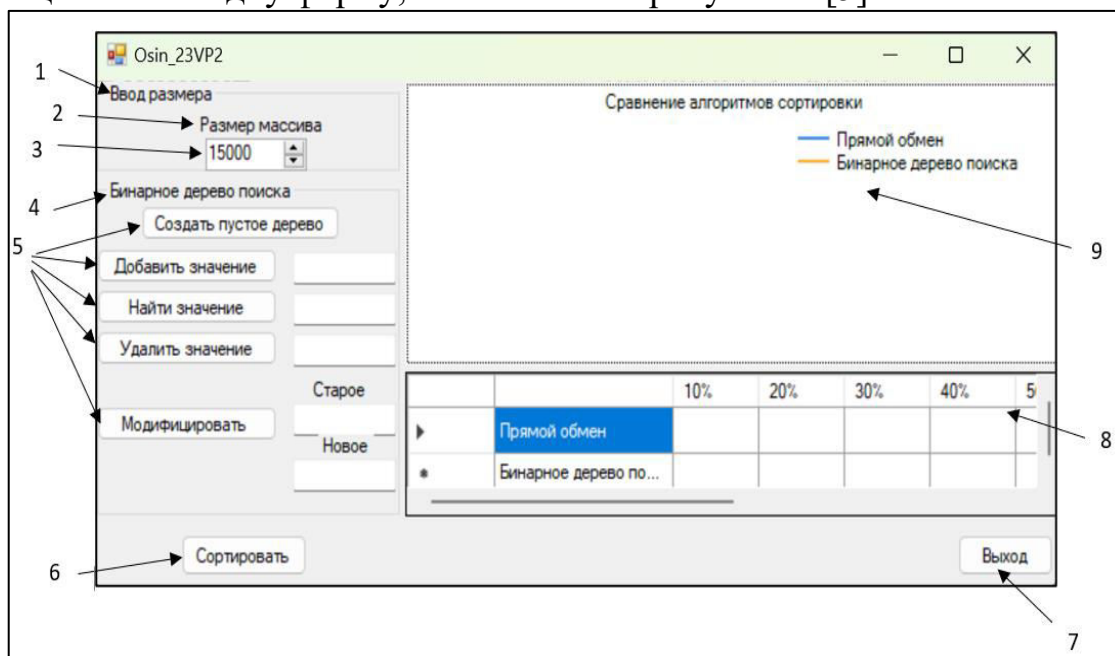


Рисунок 4 – Пользовательский интерфейс приложения

Описание обозначений на рисунке 4:

1. GroupBox: Заголовок рамки для ввода размера массива.
2. Label: Заголовок поля для ввода размера массива.
3. NumericUpDown: поле ввода размера массива в указанной последовательности (от 1 до 10000).
4. GroupBox: заголовок рамки для работы с функциями с бинарным деревом поиска.
5. Button: функции для работы с бинарными деревьями поиска.
6. Button: кнопка для сортировка массива двумя способами: прямым обменом и с помощью бинарного дерева поиска.
7. Button: кнопка завершения работы приложения.
8. DataGridView: таблица, которая заполняется при нажатии кнопки «Сортировать».
9. Chart1: график отображения время работы каждого алгоритма сортировки.

2.2 Проектирование структур данных

В курсовой работе представлены 2 структуры данных: бинарное дерево поиска и массив. Для создания неупорядоченного массива используется следующий код:

```
private void Form1_Load(object sender, EventArgs e)
{
    Random rd = new Random();
    for (int i = 0; i < SIZE; i++) array[i] = rd.Next(0, SIZE);
}
```

В данном коде в методе Form1_Load сначала объявляется класс Random с именем rd. Затем в глобальный массив array с константным размером SIZE генерируются числа от 0 до SIZE. [5]

Для создания пустого бинарного дерева поиска используется следующий код:

```
internal class BinarySearchTree
{
    public int data;
    public BinarySearchTree left;
    public BinarySearchTree right;
    public BinarySearchTree CreateTree(int data)
    {
        return new BinarySearchTree { data = data, left = null, right
= null };
    }
}
```

В данном коде описан класс BinarySearchTree с модификатором доступа internal, который означает, что класс доступен только внутри той же сборки (проекта). В классе описаны следующие атрибуты: data хранит данные в узле дерева, left – ссылка на левое поддерево текущего узла, right – ссылка на правое поддерево текущего узла. Все атрибуты с модификатором public – публичные, открытые. Также класс содержит метод CreateTree с параметром data, в котором создаётся новое бинарное дерево поиска. Сначала data присваивается как вершина дерева, а right и left объявляются пустыми (null). [5]

2.3 Структура программного обеспечения

Исходя из проектирования пользовательского интерфейса приложения, была разработана диаграмма классов, отображающая ключевые классы системы и их взаимоотношения, представлена на рисунке 5.

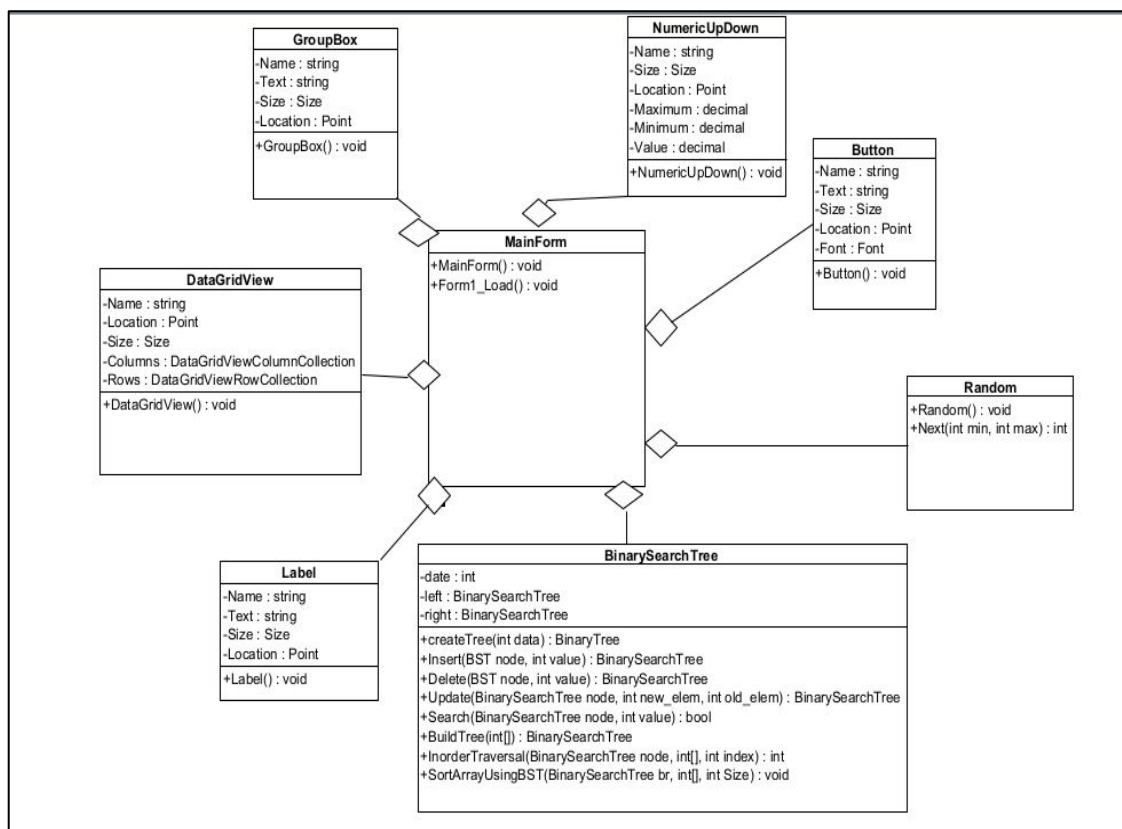


Рисунок 5 – Диаграмма классов

В диаграмме классов представлено 8 классов:

MainForm. Класс MainForm отвечает за инициализацию формы и отображение пользовательского интерфейса. Он не содержит полей, но включает в себя 2 метода:

1. MainForm (): конструктор по умолчанию.
2. Form1_Load (): обработчик события Load, которое происходит перед первым отображением формы.

Label. Класс Label – класс WinForms, который представляет собой элемент управления, предназначенный для отображения текста на форме. Основные свойства:

- Name: string – уникальное имя элемента;
- Text: string – текст, отображаемый элементом управления;
- Size: Size – определяет размер элемента управления;
- Location: Point – определяет местоположение элемента на форме.

DataGridView. Данный класс представляет собой элемент управления для отображения данных в виде таблицы. Основные свойства:

- Name: string – уникальное имя элемента;
- Text: string – текст, отображаемый элементом управления;
- Size: Size – определяет размер элемента управления;
- Columns: определяет столбцы (размер, названия, количество);
- Rows: определяет строки (размер, названия, количество).

GroupBox. Данный класс представляет элемент управления Windows, отображающий рамку вокруг группы элементов управления, а также необязательный заголовок. Основные свойства:

- Name: string – уникальное имя элемента;
- Text: string – текст, отображаемый элементом управления;
- Size: Size – определяет размер элемента управления;
- Location: Point – определяет местоположение элемента на форме.

NumericUpDown. Представляет регулятор Windows (также известный как элемент управления "вверх-вниз"), отображающий числовые значения. Свойства:

- Name: string – уникальное имя элемента;
- Size: Size – определяет размер элемента управления;
- Location: Point – определяет местоположение элемента на форме;
- Maximum: int – определяет максимальное допустимое значение, которое может ввести пользователь;
- Minimum: int – определяет минимальное допустимое значение, которое может ввести пользователь;
- Value: int – определяет текущее значение, которое видно пользователю при запуске приложения.

Button. Элемент управления, который позволяет пользователю щёлкнуть его для выполнения действия. Свойства:

- Name: string – уникальное имя элемента;
- Size: Size – определяет размер элемента управления;
- Location: Point – определяет местоположение элемента на форме;
- Text: string – текст, отображаемый элементом управления;
- Font: Font – размер кнопки и написанного в ней текста.

BinarySearchTree. Данный класс представляет собой бинарное дерево поиска, у которого 3 поля:

- Date: для хранения значения узла в бинарном дереве.
- Left: левое поддерево.
- Right: правое поддерево.

А также и методы:

- CreateTree (int data): создание пустого бинарного дерева поиска с вершиной в data;
- Insert (BinarySearchTree node, int value): вставка значения value в бинарное дерево поиска node;
- Delete (BinarySearchTree node, int value): удаление значения value из бинарного дерева поиска node;

- Update (BinarySearchTree node, int new_elem, int old_elem): изменения значения old_elem на new_elem в бинарном дереве поиска node;
- Search (BinarySearchTree node, int value): находит значение value в бинарном дереве поиска node;
- BuildTree (int[] array): строит бинарное дерево поиска из элементов массива array.
- InorderTraversal (BinarySearchTree node, int[] sortedArray, int index): инфиксный обход бинарного дерева поиска node и запись в массив sortedArray элементов по возрастанию;
- SortArrayUsingBST (BinarySearchTree br, int[] arr, int Size): сортировка массива arr размером Size с помощью бинарного дерева поиска.

Random. Класс, который генерирует псевдослучайные числа в определённом диапазоне. Обладает следующими свойствами:

- Next (int min, int max): свойства для генерации числа в последовательности от min до max.

3 Реализация программы

3.1 Кодирование

В ходе выполнения курсовой работы было разработано приложение «Разработка библиотеки функций для поддержки работы с бинарными деревьями поиска», код которого приведен в приложении А.

Для алгоритма сортировки прямым обменом была разработана диаграмма деятельности, изображенная на рисунке 6.

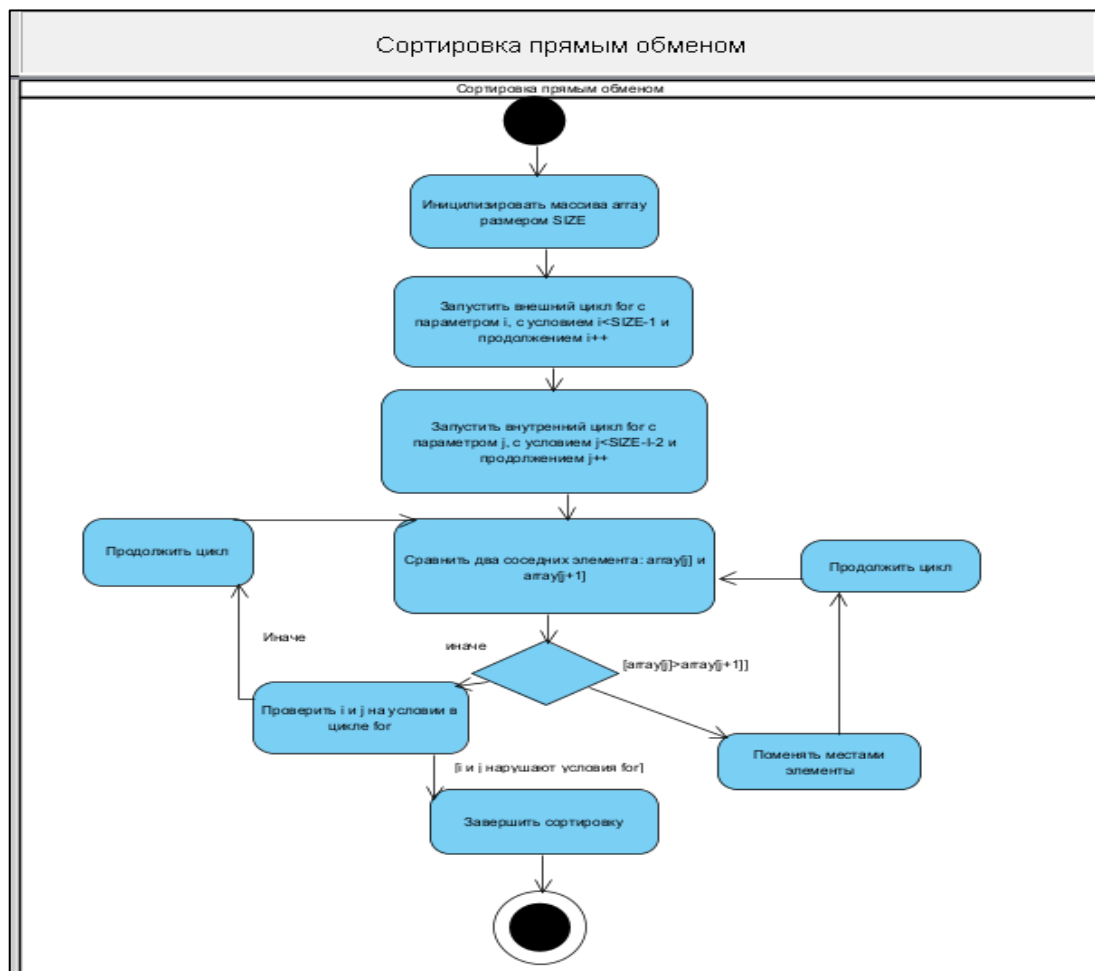


Рисунок 6 – Диаграмма деятельности

Диаграмма начинается с инициализацией массива array и его длины SIZE. Запускается внешний цикл for с параметром i=0, с условием i < SIZE-1 и продолжением i++. Запускается внутренний цикл for с параметром j=0, с условием j < SIZE-i-1 для избежания повторного прохода по уже отсортированным элементам в конце массива и продолжением j++. Далее идёт проверка (array[j] > array[j+1]), в случае истинности – элементы массива меняются местами и цикл продолжается. Иначе цикл продолжается, при условии, что i и j не нарушают условия в for. По окончании работы циклов, массив отсортирован.

3.2 Диаграмма компонентов

В процессе выполнения курсовой работы была составлена диаграмма компонентов, которая отображает разбиение программной системы на структурные компоненты и связи между компонентами, представлена на рисунке 7.

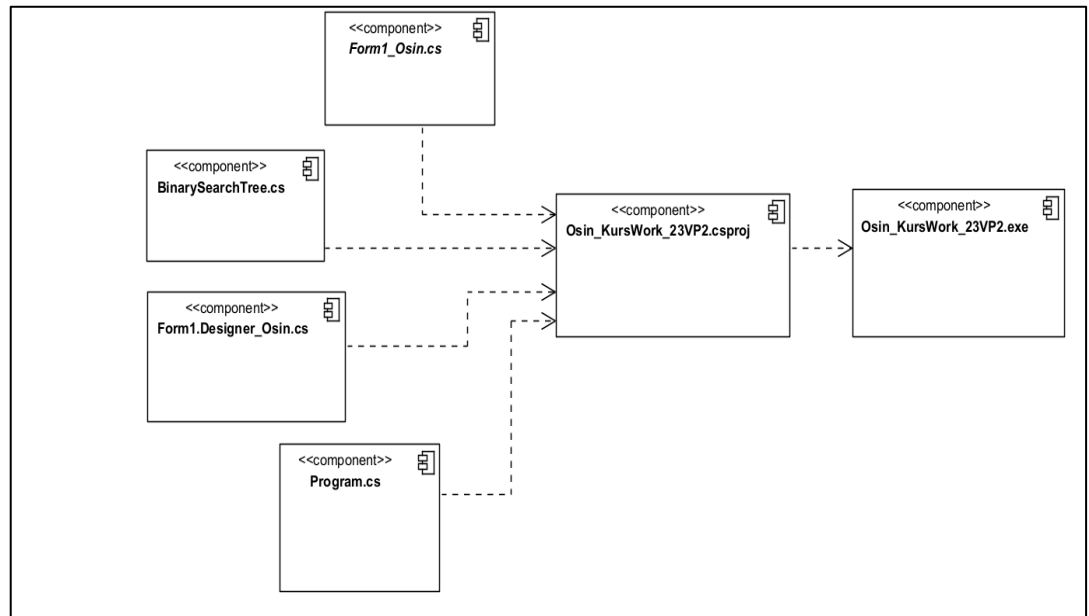


Рисунок 7 – Диаграмма Компонентов

Назначение компонентов:

- Program.cs: определяет точку входа в приложение;
- Form1.Designer_Osin.cs: содержит все элементы инициализации, связанные с WinForms;
- BinarySearchTree.cs: класс Бинарного дерева поиска, в котором содержатся все функции, описанные в техническом задании;
- Form1_Osin.cs файл класса WindowsForms, в котором записываются необходимые методы, функции, а также управляемые событиями методы и коды;
- Osin_KursWork_23VP2.csproj: файл, содержащий всё содержимое из классов;
- Osin_KursWork_23VP2.exe: исполняемый файл.

4 Тестирование

4.1 Виды тестирования программных средств

Тестирование программного обеспечения — это важный процесс, который помогает определить качество и функциональность программных продуктов. Существует множество видов тестирования, каждый из которых имеет свои цели и методы. [6]

Модульное тестирование — это процесс проверки отдельных модулей программного обеспечения на предмет их корректности.

Модуль в этом контексте представляет собой небольшую, изолированную часть программы, такую как функция, метод, класс или объект, которая выполняет определённую задачу или функцию в приложении.

Цель модульного тестирования — убедиться в том, что каждый модуль работает правильно и соответствует спецификациям. Этот метод позволяет выявить и исправить ошибки на ранних этапах разработки, что упрощает процесс отладки и обеспечивает более надёжное программное обеспечение.

Основные принципы модульного тестирования:

- **Изоляция.** каждый модуль тестируется в изоляции от остальных частей программы, чтобы убедиться, что результаты тестирования зависят только от данного модуля;
- **Автоматизация.** тесты следует автоматизировать, чтобы можно было легко повторять их выполнение при каждом изменении кода.

Функциональное тестирование — это один из ключевых видов тестирования программного обеспечения, который фокусируется на проверке функциональности системы в соответствии с её требованиями и спецификациями. Данный вид тестирования направлен на то, чтобы убедиться, что приложение выполняет все предусмотренные функции, а также работает корректно с различными входными данными. [6]

Принципы функционального тестирования:

- **Ориентация на требования:** тестирование должно основываться на функциональных требованиях, спецификациях и сценариях использования;
- **Использование реальных сценариев:** тесты должны моделировать реальные условия использования системы.

Функциональное тестирование является критически важным этапом в процессе обеспечения качества программного обеспечения. Оно помогает гарантировать, что система работает так, как задумано, и удовлетворяет потребности пользователей. Правильная организация функционального тестирования, использование эффективных инструментов и методов может значительно повысить качество конечного продукта. [6]

Регрессионное тестирование — это процесс проверки программного обеспечения, направленный на выявление дефектов, возникающих после внесения изменений в код, таких как исправления ошибок, добавление новых функций или изменения в существующих функциях. Основная цель регрессионного тестирования заключается в том, чтобы убедиться, что новые изменения не повлияли негативно на уже работающие функции системы. [6]

Цели регрессионного тестирования:

- **Выявление новых дефектов:** убедиться, что изменения в коде не привели к появлению новых ошибок в уже протестированных функциональностях;
- **Поддержка качества:** гарантировать, что система продолжает работать корректно после обновлений, что особенно важно для поддержания доверия пользователей;
- **Поддержка стабильности:** проверить, что исправления и новые функции не нарушают стабильность системы.

Регрессионное тестирование играет важную роль в процессе разработки программного обеспечения, обеспечивая стабильность и качество системы после внесения изменений. Правильная организация регрессионного тестирования, использование автоматизации и эффективных инструментов может существенно повысить эффективность процесса тестирования и снизить риски, связанные с изменениями в коде. [6]

Интеграционное тестирование — это этап тестирования программного обеспечения, который проводится после модульного тестирования и перед системным тестированием. Основная цель интеграционного тестирования заключается в проверке взаимодействия между различными модулями или компонентами системы, чтобы убедиться, что они работают корректно вместе.

Типы интеграционного тестирования:

- **Инкрементное тестирование:** компоненты интегрируются и тестируются поэтапно. Это может быть сделано по подходу "снизу вверх" (начиная с низкоуровневых модулей) или "сверху вниз" (начиная с высокоуровневых модулей);
- **Тестирование на основе интерфейсов:** проверка взаимодействия между модулями через их интерфейсы, чтобы убедиться, что данные передаются и обрабатываются корректно.

Интеграционное тестирование является критически важным этапом в процессе разработки программного обеспечения, обеспечивая корректное взаимодействие между модулями и повышая общее качество системы. Правильная организация интеграционного тестирования, использование подходящих инструментов и методов может значительно улучшить результаты тестирования и снизить риски, связанные с интеграцией компонентов.

Системное тестирование — это этап тестирования программного обеспечения, который проводится после интеграционного тестирования и перед

приемочным тестированием. Основная цель системного тестирования заключается в проверке всей системы в целом, чтобы убедиться, что она соответствует заданным требованиям и функционирует как единое целое.

Цели системного тестирования:

- **Проверка функциональности:** убедиться, что все функциональные требования, описанные в спецификациях, выполняются;
- **Проверка производительности:** Оценка быстродействия системы, ее стабильности и масштабируемости под нагрузкой;
- **Проверка безопасности:** выявление уязвимостей и проверка механизмов защиты данных;
- **Проверка совместимости:** убедиться, что система работает на различных платформах, браузерах и устройствах;
- **Проверка пользовательского интерфейса:** оценка удобства и интуитивности интерфейса для конечных пользователей.

Системное тестирование является важным этапом в процессе разработки программного обеспечения, позволяя убедиться, что система работает корректно и соответствует требованиям.

Нагрузочное тестирование — это тип нефункционального тестирования, целью которого является оценка производительности системы под определенной нагрузкой. Оно помогает определить, как система ведет себя при увеличении числа пользователей или объема данных и выявить ее пределы.

Преимущества:

- Помогает обеспечить устойчивость и надежность системы в условиях высокой нагрузки;
- Позволяет заранее выявить и устранить потенциальные проблемы с производительностью.

Нагрузочное тестирование является важным этапом в обеспечении качества программного обеспечения, особенно для веб-приложений и систем, ожидающих большого числа пользователей.

Исходя из перечисленных видов тестирования, в курсовой работе будет использовано функциональное тестирование, потому что это тестирование фокусируется на проверке того, что система выполняет все заявленные функциональные требования и позволяет выявлять серьезные ошибки и дефекты на ранних стадиях разработки.

4.2 Функциональное тестирование программы

В курсовой работе было выполнено функциональное тестирование разработанного программного обеспечения. Результаты приведены в таблице 4.

Таблица 4 – Результаты тестирования

Состав теста	Ожидаемый результат	Наблюдаемый результат
Сравнение сортировок для неупорядоченного массива размером 15000	Должна быть выполнена сортировка двумя методами, выведены результаты в табличном и графическом видах	Сортировка выполнена, результаты получены и выведены. Тест пройден. Результат представлен на рисунке 8.
Сравнение сортировок для неупорядоченного массива размером -5	Сортировка не будет выполнена по причине отрицательного размера массива. Результаты не будут выведены в табличном и графическом видах	Сортировка не выполнена, была выведена ошибка из-за отрицательного размера массива. Тест пройден. Результат представлен на рисунке 9.
Сравнение сортировок для неупорядоченного массива размером 20000	Должна быть выполнена сортировка двумя методами. Результаты будут выведены в табличном и графическом видах	Сортировка выполнена, результаты выведены в графическом и табличном видах. Тест пройден. Результат представлен на рисунке 10.
Сравнение сортировок для неупорядоченного массива размером 0	Сортировка не будет выполнена по причине нулевого размера массива. Результаты не будут выведены в табличном и графическом видах	Сортировка не выполнена, была выведена ошибка из-за нулевого размера массива. Тест пройден. Результат представлен на рисунке 11.
Сравнение сортировок для неупорядоченного массива размером 25000. В бинарное дерево поиска добавим значение 10	Должна быть выполнена сортировка двумя методами. Результаты будут выведены в табличном и графическом видах	Сортировка выполнена. Результаты выведены в табличном и графическом видах. Тест пройден. Результат представлен на рисунке 12.
Сравнение сортировок для неупорядоченного массива размером 22500. Из бинарного дерева поиска удалим значение 8	Должна быть выполнена сортировка двумя методами. Результаты будут выведены в табличном и графическом видах	Сортировка выполнена. Результаты выведены в графическом и табличном видах. Тест пройден. Результаты представлены на рисунках 13–14.

Продолжение таблицы 4

Сравнение сортировок для неупорядоченного массива размером 9000. В бинарном дереве поиска изменим значение 5 на 6	Должна быть выполнена сортировка двумя методами. Результаты будут выведены в табличном и графическом видах	Сортировка выполнена. Результаты выведены в табличном и графическом видах. Тест пройден. Результаты представлены на рисунках 15–16.
---	--	---

Ниже, на рисунках 8–16, отображены результаты работы приложения в процессе тестирования.

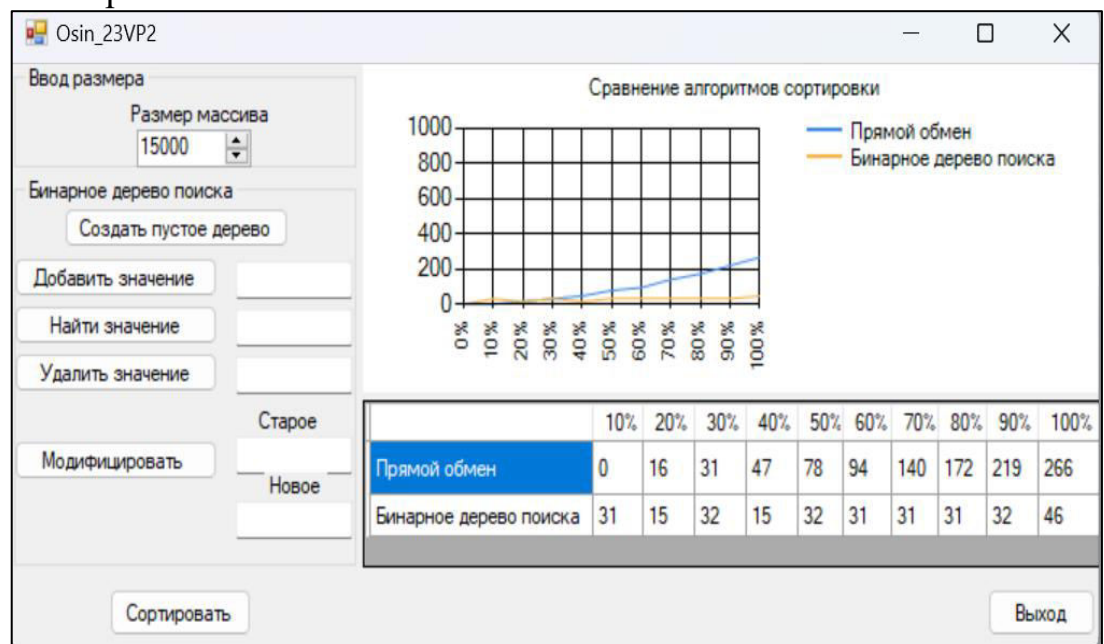


Рисунок 8 – Результаты сортировок для размера массива 15000

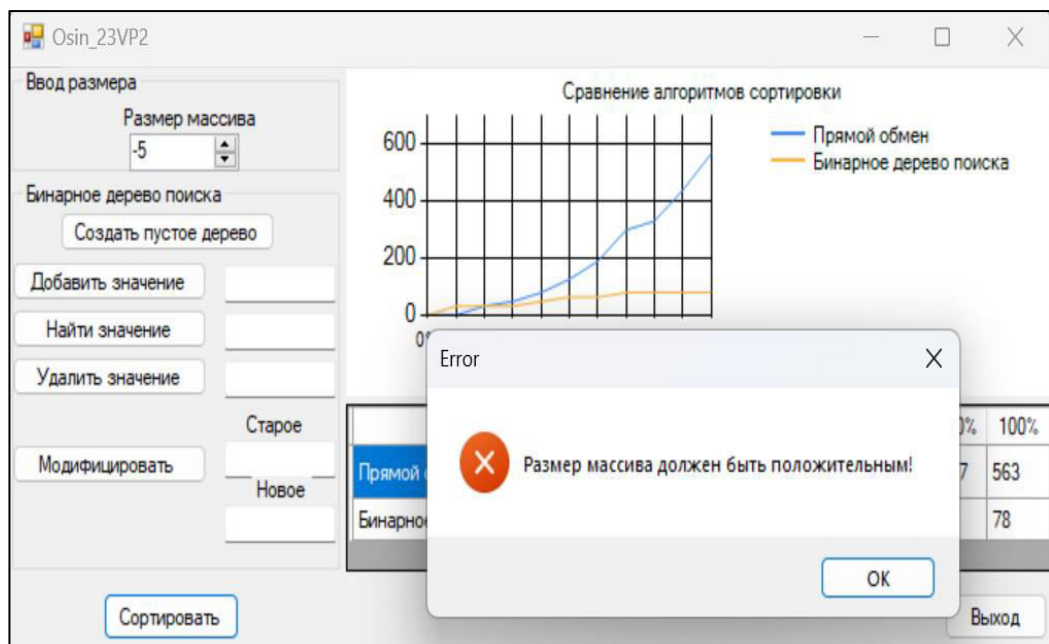


Рисунок 9 – Результаты сортировок для размера массива -5

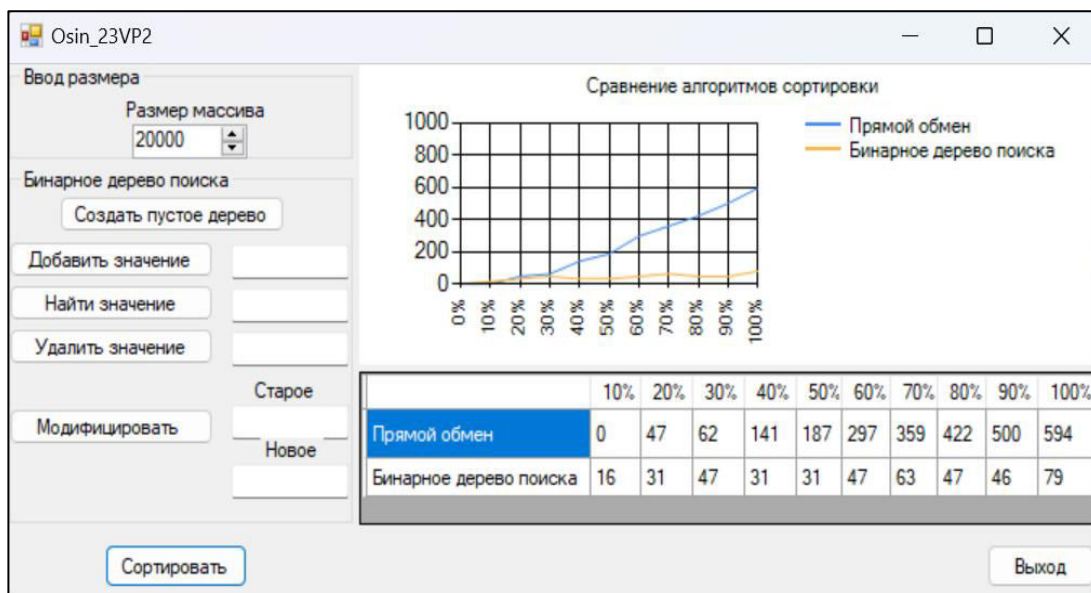


Рисунок 10 – Результаты сортировок для размера массива 20000

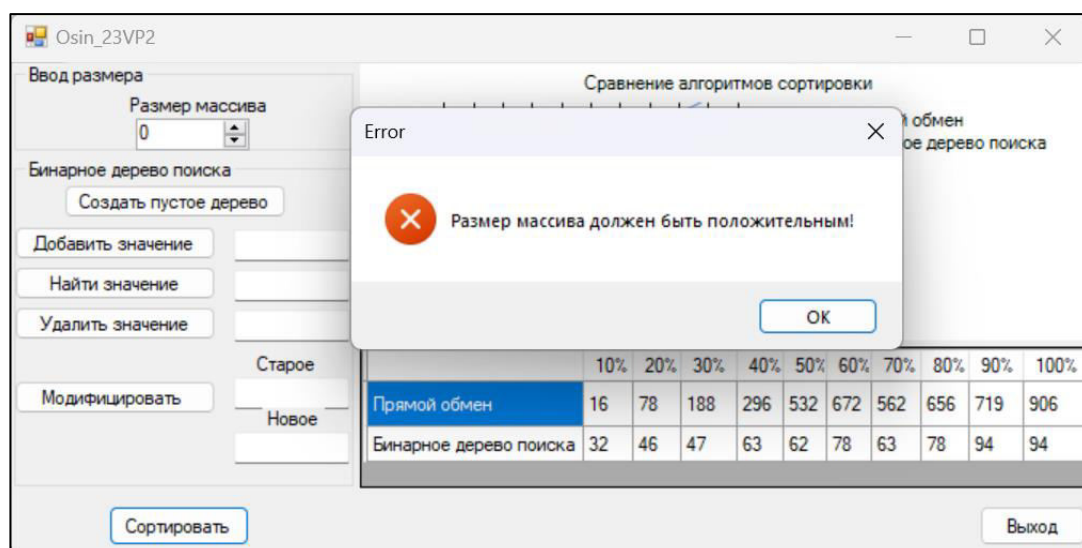


Рисунок 11 – Результаты сортировок для размера массива 0

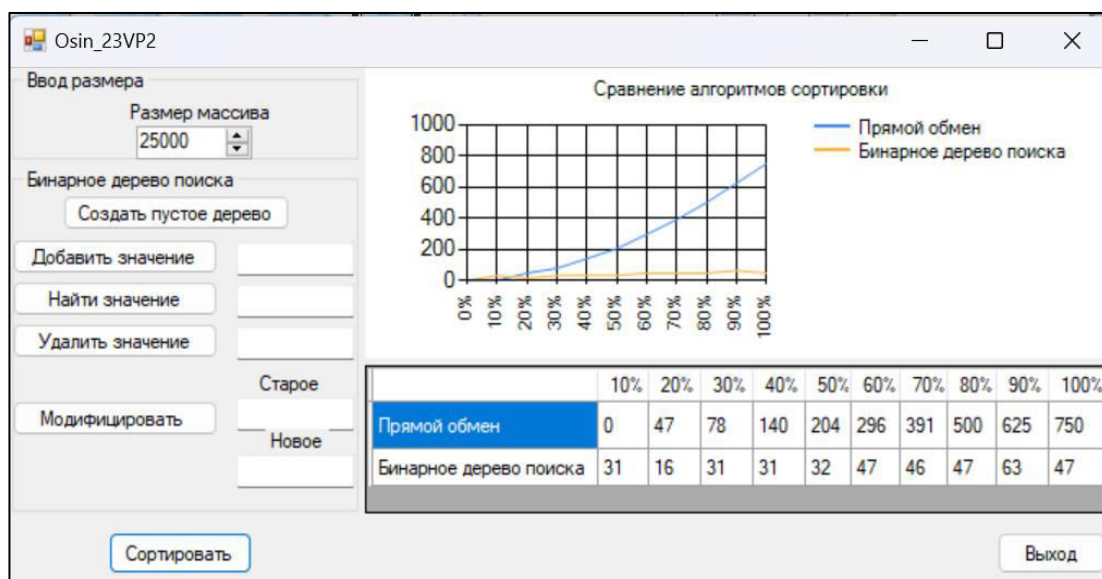


Рисунок 12 – Результаты сортировок для размера массива 25000

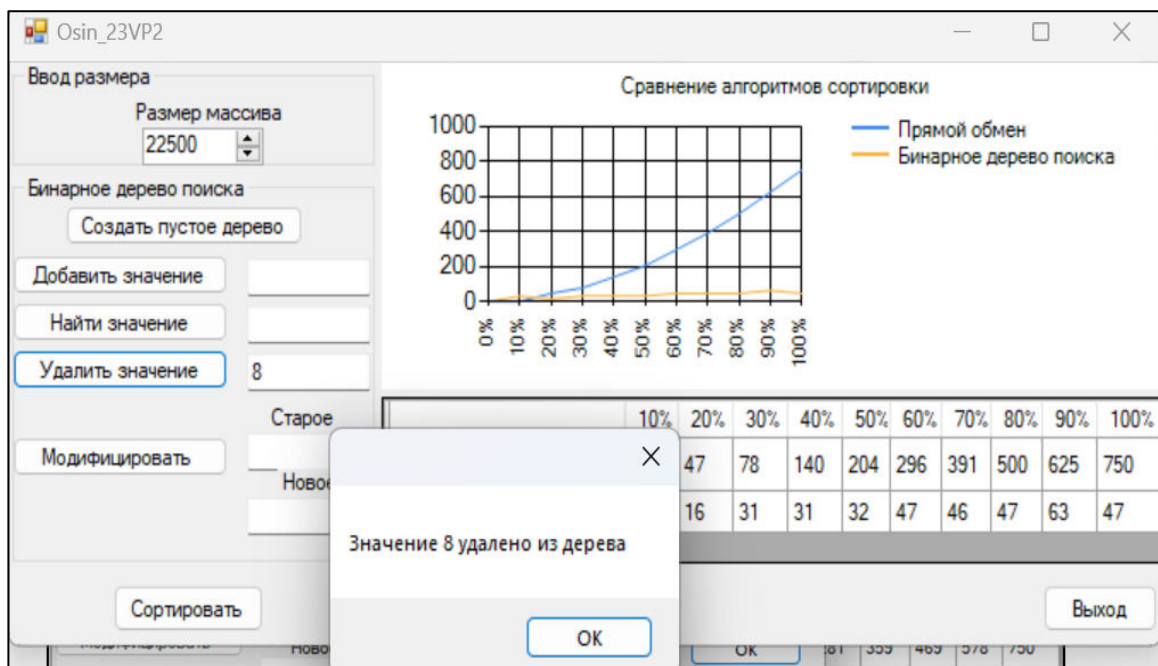


Рисунок 13 – Удаление значения 8 из бинарного дерева поиска

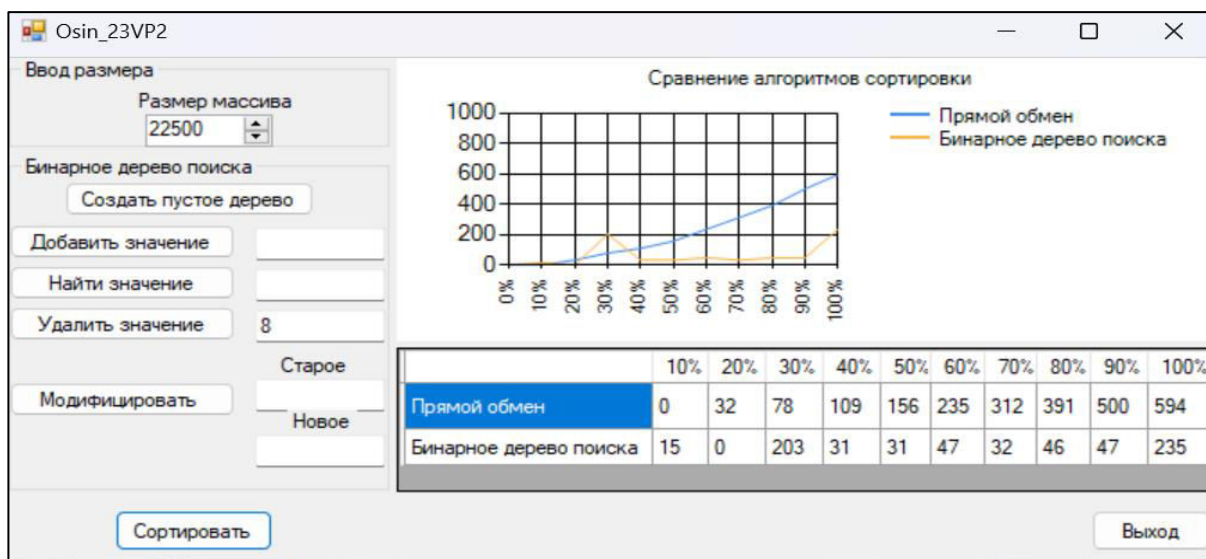


Рисунок 14 – Результаты сортировок для массива размером 22500

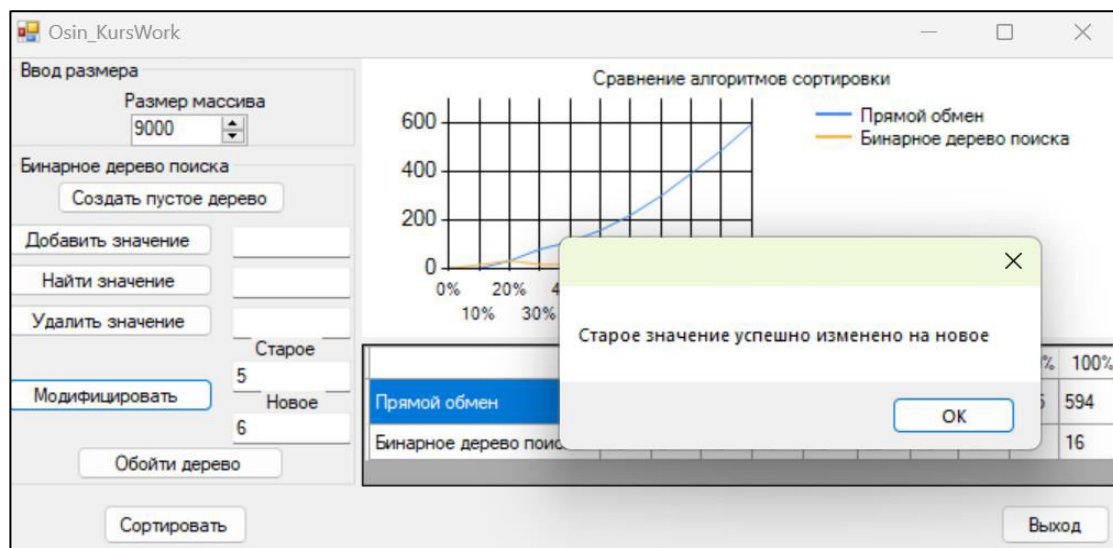


Рисунок 15 – Изменение значение 5 на 6 в бинарном дереве поиска

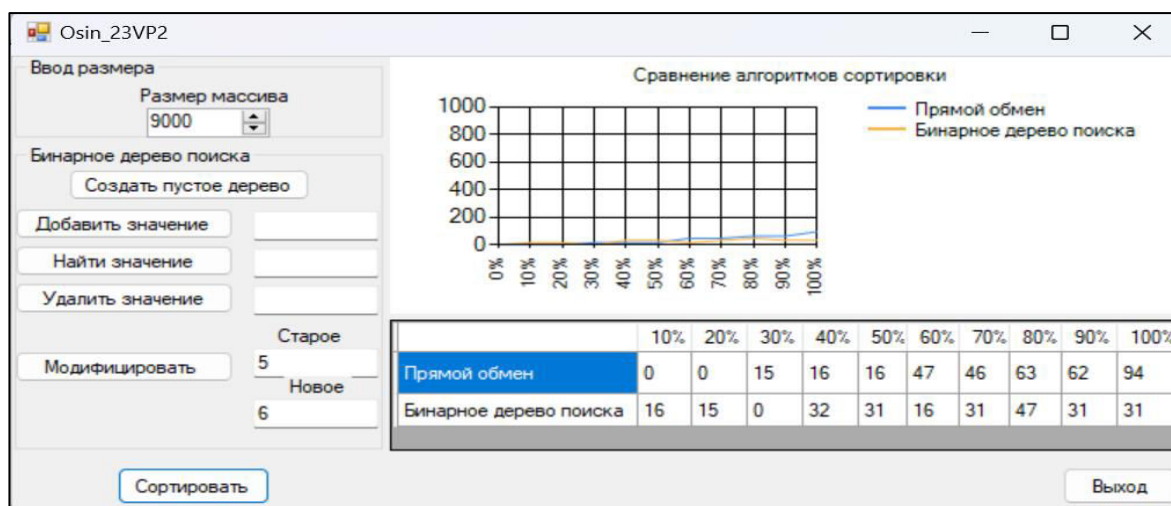


Рисунок 16 – Результаты сортировок для размера массива 9000

В ходе выполнения тестирования несовпадения ожидаемого и наблюдаемого результата не выявлены. Следовательно, можно сделать вывод, что программа работает корректно.

5 Анализ результатов

Исходя из тестирования, можно сказать, что алгоритм сортировки с помощью бинарного дерева поиска работает быстрее чем прямой обмен. Однако рассмотрим результативность данных алгоритмов сортировки для различных типов упорядоченности и размерностей сортируемой последовательности.

Пусть задан неупорядоченный массив размером 8999. Результаты работы алгоритмов сортировка приведены на рисунке 17.



Рисунок 17 – Результаты для неупорядоченного массива размером 8999.

Исходя из работы алгоритмов сортировки, можно сделать вывод, что для неупорядоченного массива эффективнее использовать алгоритм сортировки с помощью бинарного дерева поиска, потому что она позволяет простым рекурсивным способом обойти дерево от меньших элементов к большим, что выдаёт узлы дерева в возрастающем порядке и помещает их в массив.

Зададим упорядоченный по возрастанию массив размером 10000 и сравним работу алгоритмов сортировок. Вершиной бинарного дерева поиска будет срединный элемент массива. Результаты работы представлены на рисунке 18.

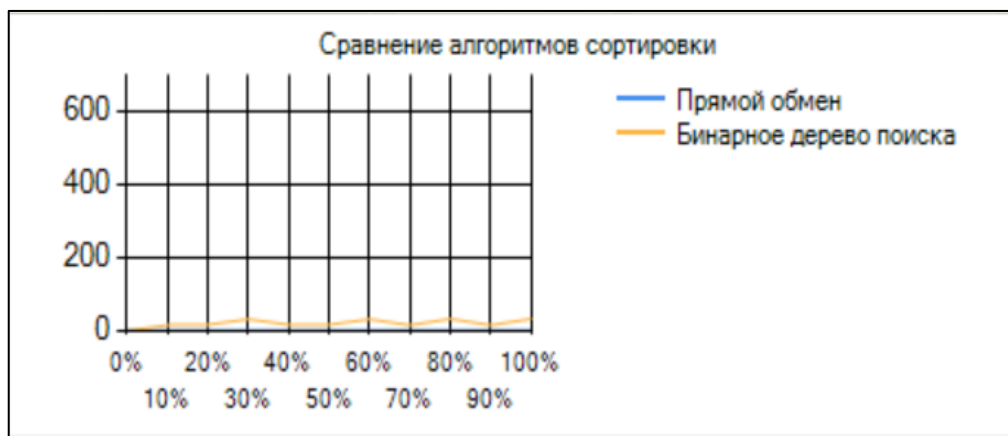


Рисунок 28 – Результаты для упорядоченного по возрастанию массива размером 10000.

Исходя из времени выполнения алгоритмов сортировки, можно сделать вывод, что если на вход подаётся упорядоченный по возрастанию массив, то выгоднее использовать сортировку прямым обменом, потому что данный алгоритм пройдет только один раз по всему массиву и не сделает ни одной перестановки, таким образом завершит свою работу.

Пусть будет дан упорядоченный по убыванию массив размером 9500. Вершиной бинарного дерева поиска будет срединный элемент массива. Результаты работы алгоритмов сортировки представлены на рисунке 19.

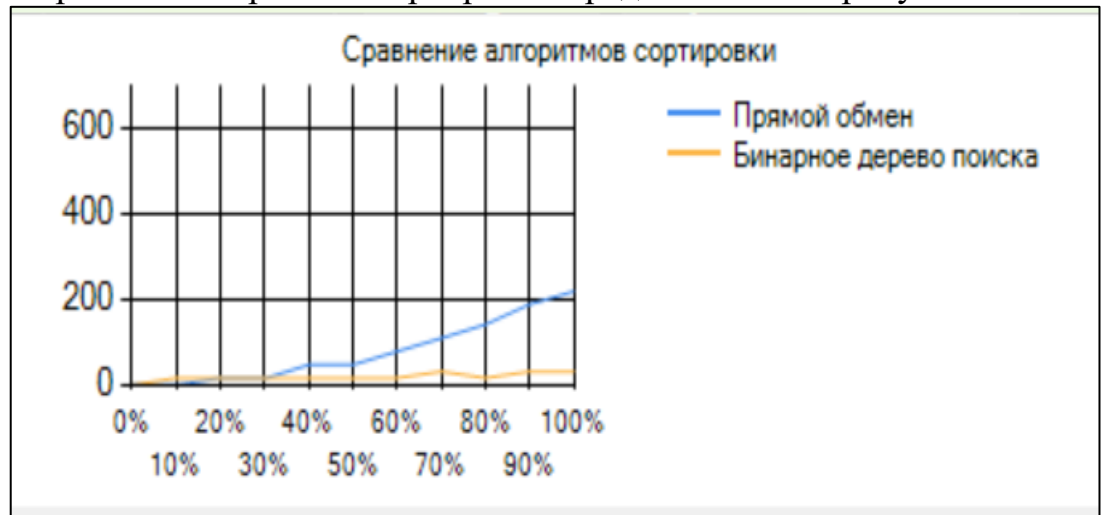


Рисунок 19 – Результаты для упорядоченного по убыванию массива размером 10000.

Исходя из полученных данных, можно сказать, что если на вход дан упорядоченный по убыванию массив, то эффективнее использовать сортировку бинарным деревом поиска, потому что при прямом обмене наибольшие элементы массива доходят до конца массива, пройдя через весь массив.

Таким образом, алгоритм сортировки прямым обменом эффективнее сортировки бинарным деревом поиска только если для сортировки будет дан уже отсортированный по возрастанию массив, во всех остальных случаях бинарное дерево поиска лучше.

Заключение

В курсовой работе была поставлена задача: разработать приложение для сравнения работы двух алгоритмов сортировки: прямым обменом и бинарным деревом поиска. Для достижения поставленной задачи работа была разбита на подзадачи для повышения эффективности и упрощения сложности разработки программного продукта.

Сначала провели анализ предметной области, а именно проанализировали структуры данных: бинарное дерево поиска и массив. Изучили основные функции каждой из структуры, затем провели анализ требований для интерфейса для удобства использования его пользователем. Спроектировали диаграмму вариантов использования для показа функционала программного обеспечения. Далее выбрали технологию разработку программного обеспечения - Agile, основываясь на некоторых факторах.

Далее была спроектирована форма, обеспечивающая функционал, описанный в техническом задании. Затем были запрограммированы структуры данных: бинарное дерево поиска и массив и спроектирована диаграмма классов для отображения ключевых аспектов системы (классов) и их взаимодействия.

Затем для сортировки прямым обмена была спроектирована диаграмма деятельности, которая описывает порядок выполнения алгоритма и диаграмма компонентов, в которой описаны файлы, используемые в проекте, и их взаимодействие.

После окончания разработки программного обеспечения был выбран один из видов тестирования – функциональное. Разработанное программное обеспечение было протестировано с помощью выбранного вида.

Тестирование было проведено на различных случаях, при которых программное обеспечение либо работало корректно, показывая на графике работу каждого алгоритма и заносая временные результаты в таблицы, либо некорректно, выдавая одну из ошибок и завершая работу приложения.

Таким образом, разработанное программное обеспечение для сравнения алгоритмов сортировок прямым обменом и бинарным деревом поиска продемонстрировало работу алгоритмов сортировок и можно сделать вывод, что сортировка бинарным деревом поиска эффективнее, чем прямой обмен во всех случаях, кроме того, когда на вход дан упорядоченный по возрастанию массив. Сравнение этих алгоритмов демонстрирует экспоненциальный рост времени работы для прямого обмена по сравнению с бинарным деревом поиска при увеличении размера входных данных.

Список использованных источников

1. Самуйлов, С.В. Структуры и алгоритмы обработки данных : учеб. пособие / С. В. Самуйлов, С. В. Самуйлова, Л. В. Гурьянов. - Пенза: Изд-во ПГУ, 2023–80 с.
2. Валединский, В. Д. Методы программирования в задачах и примерах на C/C++: учебное пособие / В. Д. Валединский, А. А. Корнев. — Москва: Издательство Московского университета, 2023–413 с.
3. Фаулер, М. UML. Основы, 3-е издание. – Пер. с англ./М. Фаулер – Санкт-Петербург: Символ - Плюс, 2004. – 192 с.
4. Мартин, Роберт. Чистый Agile. Основы гибкости. - 1-е изд./Роберт Мартин - Санкт-Петербург: Просвещение, 2020. - 256 с.
5. Лабор, В. В. Си Шарп: Создание приложений для Windows/ В. В. Лабор.— Мн: Харвест, 2003. - 384 с.
6. Пироцкая, В. Н. Основы тестирования программного обеспечения учеб. пособие / В. Н. Пироцкая, Д. А. Градусов; Владим. гос. ун-т им. А. Г. и Н. Г. Столетовых. – Владимир: Изд-во ВлГУ, 2017–100 с.

Приложение А
КОД ПРОГРАММЫ
(обязательное)

BinarySearchTree.cs

```
using System;
using System.Collections.Generic;
using System.Security.Cryptography;
using System.Windows.Forms;

namespace WindowsFormsAppl
{
    internal class BinarySearchTree
    {
        public int data;
        public BinarySearchTree left;
        public BinarySearchTree right;
        public BinarySearchTree CreateTree(int data)
        {
            return new BinarySearchTree { data = data, left =
null, right = null };
        }
        public BinarySearchTree Insert(BinarySearchTree node, int
value)
        {
            if (node == null)
            {
                return CreateTree(value);
            }
            if (value < node.data)
            {
                node.left = Insert(node.left, value);
            }
            else if (value > node.data)
            {
                node.right = Insert(node.right, value);
            }

            return node;
        }
        public BinarySearchTree Delete(BinarySearchTree node, int
value)
        {
            if (node == null)
            {
                string del = string.Format("Значение {0}
отсутствует в бинарном дереве поиска", value);
                MessageBox.Show(del, "Error",
MessageBoxButtons.OK, MessageBoxIcon.Error);
                return null;
            }

            if (value < node.data)
            {
                node.left = Delete(node.left, value);
            }
        }
    }
}
```

```

        else if (value > node.data)
        {
            node.right = Delete(node.right, value);
        }
        else
        {
            node.data = 0;
            string del = string.Format("Значение {0} удалено
из дерева", value);
            MessageBox.Show(del);
        }
        return node;
    }

    public BinarySearchTree Update(BinarySearchTree node, int
new_elem, int old_elem)
    {
        if (node == null)
        {
            MessageBox.Show("Старое значение отсутствует в
дереве", "Error", MessageBoxButtons.OK, MessageBoxIcon.Error);
            return null;
        }

        if (old_elem < node.data)
        {
            node.left = Update(node.left, new_elem, old_elem);
        }
        else if (old_elem > node.data)
        {
            node.right = Update(node.right, new_elem,
old_elem);
        }
        else
        {
            node.data = new_elem;
            MessageBox.Show("Старое значение успешно изменено
на новое");
        }
        return node;
    }

    public bool Search(BinarySearchTree node, int value)
    {
        if (node == null)
        {
            return false;
        }

        if (value == node.data)
        {
            return true;
        }
        else if (value < node.data)
        {

```



```

        return Search(node.left, value);
    }
    else
    {
        return Search(node.right, value);
    }
}
public BinarySearchTree BuildTree(int[] array)
{
    if (array.Length == 0) return null;

    BinarySearchTree bst = CreateTree(array[0]);
    for (int i = 1; i < array.Length; i++)
    {
        Insert(bst, array[i]);
    }
    return bst;
}

public int InorderTraversal(BinarySearchTree node, int[]
sortedArray, int index)
{
    if (node != null)
    {
        index = InorderTraversal(node.left, sortedArray,
index);
        if (index < sortedArray.Length)
sortedArray[index++] = node.data;
        else return index;
        index = InorderTraversal(node.right, sortedArray,
index);
    }
    return index;
}

public void SortArrayUsingBST(BinarySearchTree br,int[]
arr,int Size)
{
    if (arr == null || arr.Length == 0)
    {
        return;
    }
    int[] sortedArray = new int[Size];
    br.InorderTraversal(br, sortedArray, 0);
    return;
}
}
}

```

Form1.cs

```

using System;
using System.Collections.Generic;
using System.ComponentModel;

```

```

using System.Data;
using System.Diagnostics;
using System.Drawing;
using System.Linq;
using System.Linq.Expressions;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;
using System.Windows.Forms.DataVisualization.Charting;

namespace WindowsFormsApp1
{
    public partial class Form1 : Form
    {
        const int SIZE = 100000;
        int[] array = new int[SIZE];
        public Form1()
        {
            InitializeComponent();

            private void button1_Click(object sender, EventArgs e)
            {
                Close();
            }
            BinarySearchTree br = new BinarySearchTree();
            private void Form1_Load(object sender, EventArgs e)
            {
                numericUpDown1.Value = 15000;
                dataGridView1.RowCount = 2;
                dataGridView1.Rows[0].Cells[0].Value = "Прямой обмен";
                dataGridView1.Rows[1].Cells[0].Value = "Бинарное
дерево поиска";
                chart1.ChartAreas[0].AxisX.Minimum = 0;
                chart1.ChartAreas[0].AxisX.Maximum = 100;
                chart1.ChartAreas[0].AxisX.LabelStyle.Format = "{0}%";
                chart1.ChartAreas[0].AxisY.Maximum = 700;
                Random rd = new Random();
                int SIZE_1 = Convert.ToInt32(numericUpDown1.Value);
                for (int i = 0; i < SIZE; i++) array[i] = rd.Next(0,
SIZE);
                br.CreateTree(array[0]);
                for (int i = 1; i < SIZE; i++) br.Insert(br,
array[i]);
            }
            private void button8_Click(object sender, EventArgs e)
            {
                int SIZE_1 = Convert.ToInt32(numericUpDown1.Value);
                if (SIZE_1 <= 0)
                {
                    MessageBox.Show("Размер массива должен быть
положительным!", "Error", MessageBoxButtons.OK,
MessageBoxIcon.Error);
                }
            }
        }
    }
}

```

```

        return;
    }
    chart1.Series["Прямой обмен"].Points.Clear();
    chart1.Series["Бинарное                    дерево
поиска"].Points.Clear();

    directexchange();
    BTS();
}
private void directexchange()
{
    int SIZE_1 = Convert.ToInt32(numericUpDown1.Value);

    chart1.Series["Прямой обмен"].Points.AddXY(0, 0);
    for (int proz = 10; proz <= 100; proz += 10)
    {
        int[] b = (int[])array.Clone();
        int StartTime = Environment.TickCount;
        for (int i = 0; i < SIZE_1 * proz / 100 - 1; i++)
        {
            bool f = true;
            for (int j = 0; j < SIZE_1 * proz / 100 - 1 -
i; j++)
            {
                if (b[j] > b[j + 1])
                {
                    (b[j], b[j + 1]) = (b[j + 1], b[j]);
                    f = false;
                }
            }
            if (f)
                break;
        }
        int EndTime = (Environment.TickCount - StartTime);
        dataGridView1.Rows[0].Cells[proz / 10].Value =
EndTime;
        chart1.Series["Прямой    обмен"].Points.AddXY(proz,
EndTime);
    }
}
private void BTS()
{
    int SIZE0 = Convert.ToInt32(numericUpDown1.Value);
    chart1.Series["Бинарное                    дерево
поиска"].Points.AddXY(0, 0);
    for (int proz = 10; proz <= 100; proz += 10)
    {
        int StartTime = Environment.TickCount;
        {
            int[] newarr = (int[])array.Clone();
            br.SortArrayUsingBST(br, newarr, SIZE0 * proz
/ 100);
        }
    }
}

```

```

        int EndTime = (Environment.TickCount - StartTime);
        dataGridView1.Rows[1].Cells[proz / 10].Value =
EndTime;

        chart1.Series["Бинарное дерево
поиска"].Points.AddXY(proz, EndTime);
    }
}

private void button3_Click(object sender, EventArgs e)
{
    int elem = Convert.ToInt32(textBox1.Text);
    br.Insert(br, elem);
}

private void button2_Click(object sender, EventArgs e)
{
    br.CreateTree(array[0]);
}

private void button4_Click(object sender, EventArgs e)
{
    int elem = Convert.ToInt32(textBox2.Text);
    br.Delete(br, elem);
}

private void button6_Click(object sender, EventArgs e)
{
    int old_elem = Convert.ToInt32(textBox4.Text);
    int new_elem = Convert.ToInt32(textBox5.Text);
    if (br.Search(br, new_elem))    MessageBox.Show("Новый
элемент уже присутствует в дереве", "Error", MessageBoxButtons.OK,
MessageBoxIcon.Error);
    else br.Update(br, old_elem, new_elem);
}

private void button5_Click(object sender, EventArgs e)
{
    int elem = Convert.ToInt32(textBox3.Text);
    string str1 = string.Format("Значение {0} присутствует
в бинарном дереве поиска", elem);
    string str2 = string.Format("Значение {0} отсутствует
в бинарном дереве поиска", elem);
    if (br.Search(br, elem))
        MessageBox.Show(str1);
    else
        MessageBox.Show(str2);
    return;
}
}
}

```

Program.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace WindowsFormsAppl
{
    internal static class Program
    {
        [STAThread]
        static void Main()
        {
            Application.EnableVisualStyles();
            Application.SetCompatibleTextRenderingDefault(false);
            Application.Run(new Form1());
        }
    }
}
```